Fundaments In AI & ML

Project Report

VITyarthi

2025-2026

Name: Madhav Pillai
Reg. No.: 25BCE10351
Course Code: CSA2001
Branch: BTech. CSE(Core)
Faculty: T. Venkateshwara Rao
Date: 31 March 2026

1. Introduction

Most students have wondered at some point — "if I study this many hours, will I pass?" or "does attendance actually matter that much?" This project tries to answer questions like those using machine learning. Rather than guessing, the idea is to train a model on patterns from past data and let it make an informed estimate.

The Student Performance Predictor is a Python program that takes four inputs from the user — daily study hours, attendance percentage, sleep hours, and previous marks — and predicts what their final marks might look like. It also tells them whether that falls under Excellent, Good, Average, or Poor, and gives a few suggestions on what they could do better. The whole thing runs in the terminal, so it's lightweight and easy to use.

2. Objectives

- Build a working ML model that predicts student final marks based on study habits and lifestyle.
- Train it on a small but structured dataset covering the most relevant academic factors.
- Classify predicted marks into performance levels that are easy to understand.
- Add a suggestions feature so the output is actually useful, not just a number.
- Get hands-on practice with Python, Pandas, and Scikit-learn on a real problem.

3. Technologies Used

Technology	Version	Purpose
Python	3.x	Core programming language
Pandas	Latest	Data manipulation and DataFrame creation
Scikit-learn	Latest	Machine learning (LinearRegression)
NumPy	Latest	Numerical computing (dependency)

4. System Design & Architecture

The program works in a fairly straightforward pipeline — take input, train the model, predict, then respond:

- The user enters four values: study hours, attendance, sleep hours, and previous marks.
- These get loaded into a Pandas DataFrame along with the training data.
- A Linear Regression model is trained on the historical records.
- The model predicts marks for the user's specific inputs.
- The result is classified into a performance level and suggestions are printed out.

Data Flow:

```
User Input --> DataFrame (Pandas) --> LinearRegression.fit(X, y)
                                                |
                                LinearRegression.predict([input])
                                                |
                            Predicted Marks --> Category --> Suggestions
```

5. Dataset Description

Since this is a beginner-level project, the dataset is small and hand-crafted — just 8 student records defined directly in the code. It's not a real-world dataset, but the values are realistic enough to train a decent linear model and demonstrate the concept. The idea was to keep things simple and focus on the ML workflow rather than data collection.

Feature	Min	Max	Type	Role	Unit
study_hours	1	8	Float	Feature	hrs/day
attendance	50%	90%	Float	Feature	%
sleep_hours	5	8	Float	Feature	hrs/day
previous_marks	40	75	Float	Feature	marks
final_marks	42	80	Float	Target (y)	marks

6. Methodology

6.1 Algorithm: Linear Regression

Linear Regression was chosen because it's simple, interpretable, and well-suited to this kind of problem where the output (marks) is a continuous number. The idea is that the model learns a formula like this:

```
y = b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4
```

Where:

- y = the predicted final marks
- x1 through x4 = the four input features (study hours, attendance, sleep, previous marks)
- b0 = the base value (intercept), b1 to b4 = the weights the model figures out on its own

6.2 Training Process

Training the model is actually just a few lines of code. Here's what happens under the hood:

- Step 1: Separate the features (X) from the target variable (y) in the DataFrame.
- Step 2: Call model.fit(X, y) — this is where Scikit-learn finds the best coefficients.
- Step 3: Collect the user's input and pass it to model.predict().
- Step 4: Round the output and display it along with a performance category.

7. Code Walkthrough

7.1 Dataset Initialization

```
import pandas as pd
from sklearn.linear_model import LinearRegression

data = {
    "study_hours":    [1, 2, 3, 4, 5, 6, 7, 8],
    "attendance":     [50, 60, 65, 70, 75, 80, 85, 90],
    "sleep_hours":    [5, 6, 6, 7, 7, 7, 8, 8],
    "previous_marks": [40, 45, 50, 55, 60, 65, 70, 75],
    "final_marks":    [42, 48, 52, 58, 63, 68, 74, 80]
}
df = pd.DataFrame(data)
```

This is where the training data lives. The values are made up but follow a realistic pattern — students who study more, sleep well, and attend regularly tend to score higher. Pandas converts this dictionary into a nice table that the model can work with.

7.2 Model Training

```
X = df[["study_hours", "attendance", "sleep_hours", "previous_marks"]]
y = df["final_marks"]

model = LinearRegression()
model.fit(X, y)
```

X holds the input features, y holds the expected outputs. Calling model.fit() is basically telling Scikit-learn: 'figure out the relationship between X and y.' It handles all the math internally.

7.3 User Input Collection

```
study_hours    = float(input("Study hours per day: "))
attendance     = float(input("Attendance (%): "))
sleep_hours    = float(input("Sleep hours: "))
previous_marks = float(input("Previous marks: "))
```

7.4 Prediction

```
prediction = model.predict([[study_hours, attendance, sleep_hours,
previous_marks]])
predicted_marks = round(prediction[0], 2)
print(f"Predicted Marks: {predicted_marks}")
```

8. Output & Results

Here's what the program outputs for a typical student. The inputs used for this example were:

Input Parameter	Value
Study hours per day	5
Attendance (%)	78
Sleep hours	7
Previous marks	62

Predicted Output:

```
Model trained successfully!

Predicted Marks: 64.73
Performance Level: Good

Suggestions:
- Improve attendance
```

9. Performance Classification Logic

Predicted Marks	Performance Level	Implication
>= 75	Excellent	Outstanding performance
60 - 74	Good	Above average, minor improvements needed
50 - 59	Average	Needs consistent effort
< 50	Poor	Significant improvement required

10. Suggestions Module

This part is less ML and more plain if-else logic — but it makes the tool genuinely useful. After getting the predicted marks, the program checks the user's input values against some simple thresholds and prints out relevant advice. It's kind of like a checklist: if something looks off, call it out.

Condition	Suggestion Generated
study_hours < 4	Increase study hours
attendance < 75%	Improve attendance
sleep_hours < 6	Get proper sleep
previous_marks < 50	Revise basics and practice more
study >= 5 AND attendance >= 80 AND sleep >= 7	Keep up the good work!

11. Limitations & Future Scope

11.1 Current Limitations

- The dataset only has 8 samples, which is honestly too small for a robust model. It works for demonstration purposes but wouldn't hold up in a real-world scenario.
- Linear Regression assumes a straight-line relationship between inputs and outputs. Real academic performance is obviously more complex than that.
- There's no input validation right now — if someone types a string instead of a number, the program just crashes.
- The model is re-trained every time the program runs. There's no way to save it and reuse it later.

11.2 Future Scope

- Use a real student dataset with hundreds of records to make the predictions actually reliable.
- Try out algorithms like Random Forest or a simple neural network to see if accuracy improves.
- Build a web interface using Flask or Streamlit so it doesn't require command-line usage.
- Add proper evaluation metrics like R-squared and mean absolute error to measure how well the model is actually doing.
- Save the trained model using pickle or joblib so it doesn't need to retrain on every run.
- Add more features to the dataset — things like part-time job hours, internet access, or family background could make the predictions richer.

12. Conclusion

This project was a good first step into the world of machine learning. The goal was never to build something perfect — it was to understand how the pieces fit together: loading data, training a model, making predictions, and presenting results in a way that's actually useful to someone.

The Student Performance Predictor does exactly what it sets out to do. It takes four inputs, runs them through a trained Linear Regression model, and gives back a predicted score along with some practical advice. Yes, the dataset is small and the model is simple — but that's kind of the point. It's a working proof of concept, and with more data and a few improvements, it could genuinely be useful for students trying to understand where they stand and what they can do about it.

References

1. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

2. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

3. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

4. Python Software Foundation. Python Language Reference, version 3.x. Available at: https://www.python.org

5. Scikit-learn documentation. Linear Regression. Available at: https://scikit-learn.org/stable/modules/linear_model.html